

Programação Orientada a Objetos 2024/2025

Exercícios

Ficha Nº 8

Coleções: array, vector, set, map
(opcional: Algoritmos)

Esta ficha aborda o assunto de coleções. O termo “coleção” refere-se a uma classe de biblioteca que tem por funcionalidade principal armazenar e gerir conjuntos de dados (estes podem ser dados de tipo básico (int, char, float, etc.), objetos (instâncias de classes criadas pelo programador, string, etc.), ou até mesmo por exemplo, ponteiros para objeto). As funcionalidades principais incluem: inserir, procurar, percorrer, eliminar. As seguintes observações são importantes:

As coleções armazenam dados em número ilimitado apenas pela quantidade de memória (há exceções). Internamente, usam memória dinâmica (novamente: há exceções), mas esse aspeto é totalmente encapsulado tal como, por exemplo, acontece na classe String.

- Existem coleções de diversos tipos, havendo pequenas diferenças relativamente à forma como organizam internamente os seus dados (por exemplo: aceitam/não aceitam dados repetidos, ordenam/não ordenam os dados por uma certa ordem, etc.). Essas diferenças implicam uma maior ou menor adequação ao cenário que se tem pela frente e a escolha da coleção adequada é um assunto relevante. Nesta ficha aborda-se:
 - **array**: coleção de tamanho fixo (não usa memória dinâmica); corresponde a um *array* tradicional de C/C++ encapsulado dentro de uma classe, com alguma funcionalidade adicional e com uma lógica de iterar (percorrer) elementos semelhante às restantes coleções; estando os dados num *array* encapsulado dentro de uma classe, pode-se facilmente passar o *array* **por cópia**, coisa que não é possível com um *array* tradicional (nestes, é passado apenas o ponteiro para o primeiro elemento, ou seja, é uma lógica de passagem por referência);
 - **vector**: coleção simples de dados, armazenada internamente como *array* dinâmico - as consultas são rápidas, mas as inserções não tanto; aceita dados repetidos; não ordena os dados e mantém a ordem pela qual foram inseridos;
 - **set**: coleção simples (linear) de dados, que armazena como constantes; não aceita dados repetidos; organiza os dados por uma ordem crescente, pelo que os dados devem suportar a operação de comparação;
 - **map**: concretiza uma memória associativa na lógica chave-valor; não aceita chaves repetidas, mas pode haver duas chaves diferentes associadas a valores iguais.

Existem mais coleções e diversas variantes. Nesta ficha abordam-se apenas as que foram referidas acima.

- Cada coleção impõe um conjunto de restrições sobre os dados que vão armazenar. Estas restrições são derivadas da funcionalidade e estratégia de implementação de cada coleção:
 - **array**: tem um tamanho fixo definido à partida (cada *array* tem o seu tamanho). Tem de existir um construtor por omissão para o tipo de dados armazenados;
 - **vector**: tem de existir o construtor por cópia disponível para o tipo de dados armazenados;
 - **set**: tem de existir o operador < disponível para o tipo de dados armazenados. Se se tratar de uma classe, e a função que implementa este operador for membro, então a função tem que ser *const*;
 - **map**: para algumas das operações que o map suporta, tem de existir um construtor por omissão para o tipo dos dados armazenados.
- Todas as coleções são classes parametrizadas quanto ao tipo de dados (utiliza para isso *templates*):
 - `vetor<TIPO>`
 - `set<TIPO>`
 - `map<TIPO-da-chave, TIPO-do-valor>`
 - `array<TIPO, Tamanho-do-array>`

Iterar (percorrer) e usar os elementos armazenados numa coleção pode ser feito de várias formas. A mais poderosa é a que usa iteradores (existem funcionalidades que só estão disponíveis usando iteradores). O uso de iteradores é semelhante nas várias coleções, o que torna o seu uso mais simples e relativamente independente do tipo de coleção. Os iteradores comportam-se como ponteiros e seu uso é relativamente simples:

- Obter o iterador, o qual pertence a um tipo de dados que está definido dentro da classe da coleção:


```
Vector<XYZ> v;    // exemplo: um vector de objetos de uma classe XYZ
Vector<XYZ>::iterator it;
```
- Obter o iterador para o primeiro elemento:


```
auto/Vector<XYZ>::iterator it = v.begin();
// auto it = v.begin();
```

 É comum usar **auto** aqui para simplificar a declaração da variável
- usar o objeto/dado apontado pelo iterador


```
*it;    // coisa apontada pelo iterador (coisa armazenada na coleção – o objeto de XYZ)
it->funcaoMembroDaCoisaArmazenada();    // função membro da classe XYZ
```
- Movimentar o iterador para a próxima posição, mas evitando usar o iterador para o “último” elemento, o qual é, na verdade um elemento fictício que está uma posição para lá do último elemento (trata-se de uma sentinela – se já atingiu esta posição, a coleção já acabou)


```
while (it != v.end() ) {
    // ...
    ++it;    // avança para o elemento seguinte
}
```

1. Listas de compras – Várias versões

Este exercício compreende várias versões da mesma funcionalidade. Não pode usar *arrays* clássicos de C/C++ nem memória dinâmica. Existe uma classe dada como ponto de partida, que pode modificar à medida que vai fazendo as alíneas, mas apenas se fizer sentido e justificada a alteração.

Ponto de partida: considere a classe **Compra** apresentada abaixo. Representa itens a comprar.

Compra.h

```
// includes e outras declarações omitidos

class Compra {
public:
    Compra(string n, int q);
    string getNome() const;
    int getQty () const;
    void atualizaQty(int dif);
    string descricao() const;
private:
    string nome;
    int qty;
};
```

Compra.cpp

```
// includes e outras declarações omitidos
#include "Compra.h"

Compra::Compra(string n, int q) : nome(n), qty (q > 0? q : 0) {}
string Compra::getNome() const { return nome; }
int Compra::getQty() const { return qty; }
void Compra::atualizaQty(int dif) {
    qty += dif;
    if (qty < 0)
        qty = 0;
}
string Compra::descricao() const {
    return nome + ": " + to_string(qty);
}
```

Lista de compras

Pretende-se que, ao longo das alíneas seguintes, construa uma classe que representa uma lista de compras de tamanho fixo com a seguinte funcionalidade:

- Inserir uma nova coisa a comprar dado o nome da coisa e a quantidade. Se a coisa já estiver na lista, deve acrescentar a quantidade ao que já lá estiver.
- Remover uma quantidade a comprar de uma determinada coisa. Se a quantidade chegar a zero, remova-a.
- Eliminar uma coisa a comprar dado o seu nome.
- Obter a lista de coisas a comprar, uma coisa (e quantidade) por linha. Obviamente, a classe não imprime nada para o ecrã.
- Inicialmente o conjunto de coisas a comprar está vazio.

Pode usar como base de trabalho a classe **ListaCompras** cujo .h incompleto se apresenta a seguir. Cada alínea introduz uma pequena variação, pelo que deve fazer várias classes para a lista de compras.

ListaCompras.h

```
// includes e outras declarações omitidos

class ListaCompras {
public:
    bool adiciona(string nome, int qty);
    bool removeQty(string nome, int qty);
    bool elimina(string nome);
    string obtemLista() const;
private:
    // Uma coleção do tipo apropriado à alínea que está a fazer
    // em princípio, não precisa disto: int quantos;
};
```

- a) Implemente uma lista de compras de tamanho fixo. O tamanho é 20. Antes de começar a escrever código, responda às seguintes questões:
- i) Qual a coleção mais adequada (a que responde aos requisitos sem introduzir *overheads* desnecessários)? Sugestão: não se pretende usar um vector nesta alínea.
 - ii) Existe a necessidade de representar um item de Compra “vazio”? Como vai representar esse caso?

- b) Implemente uma lista de compras que não tem tamanho fixo, podendo estar realmente vazia (sem item nenhum), e cujo tamanho máximo depende apenas da memória disponível. Antes de começar a escrever código, responda às seguintes questões:

- i) Qual a coleção mais adequada (a que responde aos requisitos sem introduzir *overheads* desnecessários)?
- ii) Não é suposto ser necessário representar um item Compra “vazio”. Verifique que não tem coisas desnecessárias na classe ListaCompra e na classe Compra.

Deve implementar também a funcionalidade de eliminar todos os itens que têm uma determinada quantidade indicada. Ou seja, deve acrescentar (e implementar) o seguinte método à classe ListaCompras:

```
int eliminaTodosCom(int qty); // elimina todos os que têm quantidade qty
// retorna o número de elementos apagados
```

- c) Implemente uma lista de compras semelhante à que fez na alínea b, mas com a seguinte particularidade: os itens a comprar estão sempre ordenados por nome (ordem alfabética), mesmo que os insira por outra ordem. Antes de começar a escrever código, reflita no seguinte:
- i) Qual a coleção mais adequada (a que responde aos requisitos sem introduzir *overheads* desnecessários e lhe poupa trabalho)?

No final do exercício deverá ter

- Usado coleções para armazenar objetos – array, vector, set
- Experimentado as operações básicas em coleções: inserir, procurar, eliminar, iterar
- Ter usado iteradores

2. Opcional (TPC / Desafio) – Algoritmos

Escreva uma nova versão do exercício 1 usando o mais que puder os algoritmos da biblioteca standard e expressões lambda. Este exercício visa promover a iniciativa e o interesse, sendo acompanhado apenas por solicitação daqueles que estiverem interessados em o realizar.

- Aceda à lista de algoritmos standard e identifique os algoritmos que podem ser úteis (por exemplo: procurar, contar, eliminar);
- Veja os requisitos desses algoritmos e a descrição exata daquilo que realmente fazem;
- Aplique-os, usando expressões lambda onde necessário.

No final do exercício deverá ter

- Usado algoritmos standard e expressões lambda no contexto de coleções

3. Agenda de contactos – sem arrays nem memória dinâmica

Pretende-se implementar uma classe Agenda que representa uma vulgar agenda de contactos (funcionalidade exata descrita mais abaixo). O contacto é representado pela classe já fornecida, cujo código se encontra abaixo.

Contacto.h

```
// includes e outras declarações omitidos

class Contacto {
public:
    Contacto(string n, string m, int tel);
    string getNome() const;
    string getMorada() const;
    int getTelef() const;
    void mudaMorada(string m);
    void mudaTelef(int tel);
    string descricao() const;
private:
    string nome;
    string morada;
    int telef;
};
```

Contacto.cpp

```
// includes e outras declarações omitidos
#include "Contacto.h"

Contacto::Contacto(string n, string m, int tel) : nome (n), morada(m), telef(tel) {}
string Contacto::getNome() const { return nome; }
string Contacto::getMorada() const { return morada; }
int Contacto::getTelef() const { return telef; }
void Contacto::mudaMorada(string m) { morada = m; }
void Contacto::mudaTelef(int tel) { telef = tel; }
string Contacto::descricao() const {
    return nome + " " + morada + " " + to_string(telef);
}
```

A agenda a implementar deve associar nomes aos respetivos contactos da forma mais direta possível. Deve ter a seguinte funcionalidade (use o código fornecido mais abaixo como ponto de partida):

- Permite inserir um novo contacto, ficando associado a esse nome;
- Permite eliminar um contacto dado o seu nome;
- Permite obter o telefone associado a um determinado nome;
- Permite obter a morada associada a um determinado nome;
- Permite atualizar o telefone associado a um determinado nome;
- Permite atualizar a morada associado a um determinado nome;
- Permite mudar o nome, mantendo os restantes dados, dado o nome antigo e o novo nome;
- Permite obter a lista completa dos contactos, um contacto por linha, e não se trata de imprimir coisas no ecrã.

Agenda.h

```
// includes e outras declarações omitidos

class Agenda {
public:
    bool insere(string nome, string morada, int tel);
    bool elimina(string nome);
    int obtemTelefone(string nome) const;
    string obtemMorada(string nome) const;
    bool atualizaTelefone(string nome, int tel);
    bool atualizaMorada(string nome, string morada);
    bool atualizaNome(string nomeAntigo, string nomeNovo);
    string obtemLista() const;
private:
    // Uma coleção do tipo apropriado à alínea que está a fazer
    // em princípio, não precisa disto: int quantos;
};
```

No final do exercício deverá ter

- Usado a coleção map
- Experimentado as operações básicas em maps: inserir, procurar, eliminar, iterar
- Ter usado iteradores em maps

4. (TPC) RegistoCivil / Clube – sem memória dinâmica nenhuma

Recorde os exercícios da ficha anterior (número 7) relativos ao RegistoCivil e Clube. Nesses exercícios usou memória dinâmica para ter, no RegistoCivil: um *array* dinâmico de ponteiros para Pessoa (a população); no Clube: um *array* dinâmico de ponteiros para Pessoa (os sócios). Pretende-se eliminar a memória dinâmica, substituindo os *arrays* dinâmicos por coleções.

-> Refaça os exercícios Registo Civil e Clube usando coleções. Faça uma cópia do projeto que já tem resultante da ficha anterior e faça a nova versão pretendida.

Pode usar a versão que entender (com ponteiros *raw* ou com *smartpointers*), mas a que faz mais sentido é a mais recente (*smartpointers*). Para maior comodidade, reproduz-se aqui o texto do enunciado relativo a esses exercícios da versão *smartpointers* (da ficha 7).

Classe auxiliar - Pessoa – Classe de apoio

Esta classe não constitui exercício em si mesma e serve para apoio aos dois exercícios seguintes. **Não pode acrescentar código a esta classe.**

Pessoa.h

```
// includes e outras declarações omitidos
class Pessoa {
public:
    Pessoa(string nome, int bi, int nif);
    string getNome() const;
    int getBI() const;
    int getNIF() const;
    void setNome(string nome);
    string descricao() const;
private:
    string nome;
    int bi, nif;
};
```

Classe *Pessoa*, descrita por nome, número de bilhete de identidade (BI), e número de contribuinte (NIF), sendo necessário indicar todos estes dados na sua inicialização. Permite obter cada um dos seus dados, e permite atualizar o nome. Permite obter uma *string* com a descrição do seu conteúdo.

Pessoa.cpp

```
// includes e outras declarações omitidos#include "Pessoa.h"

Pessoa::Pessoa(string _nome, int _bi, int _nif) : nome(_nome), bi(_bi), nif(_nif){}
string Pessoa::getNome() const { return nome; }
int Pessoa::getBI() const { return bi; }
int Pessoa::getNIF() const { return nif; }
void Pessoa::setNome(string _nome) { nome = _nome; }
string Pessoa::descricao() const {
    ostringstream oss;
    oss << nome << " " << bi << " " << nif;
    return oss.str();
}
```

Registo Civil de pessoas com smartpointers – Composição de recursos dinâmicos com coleções

Escreva uma classe **RegistoCivil** semelhante às versões já feitas neste tema (ficha 6), mas com algumas simplificações e pequenas diferenças. Reproduz-se aqui o essencial, com as diferenças assinaladas em **itálico negrito**:

- Na sua inicialização exige o nome do país. Inicialmente não tem pessoa nenhuma.
- É possível obter, mas não mudar o nome do país.
- **Não existe um número máximo de pessoas fixo.**
- Permite adicionar uma nova pessoa, a qual é construída no contexto da classe e não fora dela. Para esta operação são indicados os dados da nova pessoa. A operação falha se já existir nesse país uma pessoa com o BI indicado.
- Permite obter o nome de uma pessoa dado o seu BI.
- Permite obter a listagem de todas as pessoas numa *string*, uma pessoa por linha.
- Permite atualizar o nome de uma pessoa dado o seu BI e o novo nome.
- Permite obter o número de pessoas atualmente existentes.
- Permite obter o ponteiro para uma pessoa dado o BI. **Não pode ser um ponteiro *raw*.** Este ponteiro irá ser usado por outras classes de forma permanente segundo a lógica de agregação, permanecendo o controlo exclusivo no lado do RegistoCivil, pelo que **é importante que a escolha do tipo de ponteiro é importante.**
- Apagar uma pessoa dado o BI. **Se o resto do programa estiver na posse de um ponteiro para esta pessoa, esse mesmo resto de programa deverá poder perceber que a pessoa já não existe.**
- **Transferir uma pessoa para outro registo civil, dados os BI e o registo civil destino. A operação falha se no registo civil destino já existir esse BI.**

Clube com smart pointers – Agregação por *smart pointers*

Escreva classe que representa o conceito de *clubes de futebol* em C++. Nesta versão, a funcionalidade é exatamente a mesma, mas a implementação **recorre a coleções de forma a suportar um número ilimitado de sócios.**

A classe deve ter a seguinte funcionalidade:

- Inicialização dado o nome da atividade do clube. Inicialmente fica sem sócio nenhum.
- Adicionar um sócio dado o ponteiro para essa pessoa. Não pode haver sócios repetidos (mesmo BI).
- Remover um sócio dado o seu BI.
- Obter a lista com os nomes de todos os sócios (um nome por linha)

A classe deve estar preparada para que os seus objetos se comportem corretamente no contexto da linguagem C++.

No final do exercício deverá ter

- Usado coleções para armazenar ponteiros